Ad³智投平台Android SDK 接入说明

前置说明

Ad³智投广告平台是独立运营的三方广告聚合平台，致力于帮助开发者精耕流量价值。
我们是聚合平台同时，既拥有预算侧（Ad³ADX）及开发者侧，多重的运营经验，
在提升营收的同时，更注重用户体验，更懂开发者需要什么。
【聚焦口碑传播和长期主义，从开发者应用的商业化视角与开发者进行长期稳固的合作。】

参数申请

目前Ad³智投广告SDK需要的APPID和各广告位对应ID，需要在Ad³平台申请或联系具体负责人！

支持的广告和广告类型

广告	开屏广告	原生广告	模板广告	横幅广告	插屏广告	全屏视频广告	激励视频广告
穿山甲	✔	✔	✔	✔	✔	✔	✔
优量汇	✔	✔	✔	✔	✔	✔	✔
快手	✔	✔	✔	✕	✔	✔	✔
京准通	✔	✔	✕	✕	✔	✕	✕
百度联盟	✔	✔	✔	✔	✔	✔	✔
Sigmob	✔	✔	-	✕	✔	✔	✔
趣盟	✔	✔	-	-	✔	-	✔
MTG	✔	✔	✔	✔	✔	✔	✔
Adhub	✔	-	✔	✔	✔	✔	✔
华为	✕	✕	-	✔	✔	-	✔
OPPO	✕	✔	✔	✔	✔	✔	✔
米盟	✔	✔	✔	✔	✔	✔	✔

AD平台配置

接入方需事先在Ad³平台申请相关广告参数进行配置，才能正常使用本SDK的聚合功能。

一、准备工作

1.SDK 导入依赖

app目录下的build.gradle文件添加依赖

```
android {
    ...
    defaultConfig {
        ...

        // 向下兼容版本为24
        minSdkVersion 24
        ...
    }
}

// 引用 aar
repositories {
    flatDir {
        dirs 'libs'
    }
}

dependencies {
    // 广告
    implementation fileTree(include: ['*.jar', '*.aar'], dir: 'libs_yd/ad')
  //如需使用华为SDK，通过此远程依赖引入
  implementation 'com.huawei.hms:ads-lite:13.4.74.302'

  //美数(汇盈)需要，如使用美数(汇盈)需要引用
  implementation 'com.squareup.okhttp3:okhttp:3.12.1'
  implementation 'com.google.code.gson:gson:2.8.5'
  implementation 'com.googlecode.android-query:android-query:0.25.9'
  implementation 'com.android.support:cardview-v7:27.1.1'

  //优推需要，如使用优推需要引用
  implementation 'com.squareup.okhttp3:okhttp:3.12.1'
  implementation 'com.squareup.okhttp3:logging-interceptor:3.12.0'
  implementation 'com.google.code.gson:gson:2.8.5'
  implementation 'com.github.bumptech.glide:glide:4.8.0'
  implementation 'com.google.android.exoplayer:exoplayer-core:2.13.3'
  implementation 'com.google.android.exoplayer:exoplayer-ui:2.13.3'
  implementation 'commons-codec:commons-codec:1.15'

  //快手需要，如使用快手需要引用
  implementation 'com.android.support:design:28.0.0'
  implementation 'com.android.support:recyclerview-v7:28.0.0'

  //oppo需要，如使用oppo广告需要引用
  implementation 'androidx.appcompat:appcompat:1.0.2'//注意500版本之后强要求依赖
  implementation 'androidx.recyclerview:recyclerview:1.0.0'//注意500版本之后强要求
依赖
  implementation 'androidx.legacy:legacy-support-v4:1.0.0'
  implementation 'com.squareup.okio:okio:2.5.0'
  implementation 'org.jetbrains.kotlin:kotlin-android-extensions-runtime:1.3.72'
```

```
  implementation 'android.arch.persistence:db-framework:1.1.1'//410版本新增
  implementation 'androidx.palette:palette:1.0.0'//490版本新增
  //由于可能与接入业务产生冲突，故由开发者决定依赖的版本，wechat sdk区分wechat-sdk-android与wechat-sdk-android-without-mta,由开发者决定依赖版本
  implementation "com.tencent.mm.opensdk:wechat-sdk-android-without-mta:5.5.8"//注意480版本新增，可以提升广告效果

  //米盟需要，如使用米盟广告需要引用
  implementation 'com.google.code.gson:gson:2.8.5'
 }
```

2.权限和架构说明

2.1权限

SDK所需权限如下：

```
<!--SDK通用 必要权限-->
<uses-permission android:name="android.permission.INTERNET" />
<uses-permission android:name="android.permission.ACCESS_NETWORK_STATE" />
<uses-permission android:name="android.permission.ACCESS_WIFI_STATE" />
```

2、以下权限为可选权限，申请以下权限用于防作弊功能以及有助于广告平台投放广告

```
<!--可选权限，申请后用于防作弊功能以及有助于广告平台投放广告-->
<uses-permission android:name="android.permission.READ_PHONE_STATE" />
<uses-permission android:name="android.permission.WRITE_EXTERNAL_STORAGE" />
<uses-permission android:name="android.permission.READ_EXTERNAL_STORAGE" />
<uses-permission android:name="android.permission.REQUEST_INSTALL_PACKAGES" />
<uses-permission android:name="android.permission.ACCESS_FINE_LOCATION" />
<uses-permission android:name="android.permission.ACCESS_COARSE_LOCATION" />
<uses-permission android:name="android.permission.ACCESS_FINE_LOCATION" />
<uses-permission android:name="android.permission.ACCESS_COARSE_LOCATION" />
```

2.2架构

部分广告平台的SDK中带有so库，各平台使用的so文件支持的架构如下：（开发者按需选择以下架构）

穿山甲：arm64-v8a、armeabi-v7a

快手广告：armeabi、arm64-v8a、armeabi-v7a、x86_64、x86 (3.3.15后不支持armeabi)

建议：在build.gradle中添加如下配置，可基本支持大部分手机架构：

```
defaultConfig {
    ...
    ndk{
        abiFilters 'armeabi-v7a','arm64-v8a'
    }
}
```

3. 代码初始化

请在用户授权后在合适的位置进行初始化，请求广告前必须先执行初始化。

```
public class MyApplication extends Application {
```

```
        @Override
        public void onCreate() {
            super.onCreate();
             // 参数1：上下文 Context
             // 参数2：appid，AD平台创建应用时获取，必填
            YdConfig.getInstance().init(this,appid);
             // 参数1：上下文 Context
             // 参数2：appid，AD平台创建应用时获取,必填
             // 参数3：渠道ID String，和后台对应渠道后可以分渠道配置流量，如不需要传空字符串
             // 参数4：是否开启Debug Boolean，建议在开发阶段打开Debug，方便定位问题。正式
环境请关闭
             //YdConfig.getInstance().init(this,appid, "渠道id", false);
             ...
        }
    }
```

4.SDK说明

aar名称	作用
AD_1.0.x	AD核心包，必选
AD_adapter_bd_1.0.x	百度适配器，如使用百度广告必选
AD_adapter_jd_1.0.x	京准通适配器，如使用京准通广告必选
AD_adapter_ks_1.0.x	快手适配器，如使用快手广告必选
AD_adapter_tt_1.0.x	穿山甲适配器，如使用穿山甲广告必选
AD_adapter_tx_1.0.x	优量汇适配器，如使用优量汇广告必选
AD_adapter_sb_1.0.x	Sigmob适配器，如使用Sigmob广告必选
AD_adapter_qtt_1.0.x	趣盟适配器，如使用趣盟广告必选
AD_adapter_mtg_1.0.x	MTG适配器，如使用MTG广告必选

二、广告接入

1. 开屏广告

展示开屏页的位置：冷启动进入应用时，加载开屏页广告，广告展示完毕，根据各应用的逻辑跳转到相应页面。建议将应用的logo放到广告页的下面，高度不超过200个像素（详见demo），或者将logo页作为广告的背景，有广告返回时广告盖住logo，无广告返回时继续展示logo。

特别注意:当用户点击广告跳转到广告详情页面后onAdclose()方法仍然会被调用，此时开发者还不能打开自己的App主页，当从广告落地页返回以后才可以跳转。所以使用canJump字段配合页面生命周期辅助判断正确的跳转时机。

接入代码示例

```java
private YdSpread ydSpread;
private boolean canJump;
/**
* 加载开屏页
*
* context 上下文
* setKey 运营分配的媒体位
* setScreenPercentage 开屏广告展示区域占屏幕高度的比例，不设置默认为1即全屏高度。
*                     取值范围为0.75到1。如果设计的开屏不是铺满全屏需要计算此值传入。
* setSpreadLoadListener 请求成功监听
* setSpreadListener 回调监听
*/
private void loadSplashAd() {
        ydSpread = new YdSpread.Builder(this)
                .setKey(String)
                .setScreenPercentage(1)//此方法可不调用，具体说明见上面注释
                .setSpreadLoadListener(new SpreadLoadListener() {
                    @Override
                    public void onADLoaded(final SpreadAd spreadAd) {
                        // 广告请求成功回调该方法
                        spreadAd.show(mAdContainer);
                    }
                })
                .setSpreadListener(new AdViewSpreadListener() {
                    @Override
                    public void onAdDisplay() {
                        // 广告展示时回调该方法
                    }

                    @Override
                    public void onAdFailed(YdError error) {
                        // 广告异常、失败，回调该方法
                    }

                    @Override
                    public void onAdClick() {
                        // 广告被点击时回调该方法
                    }

                    @Override
                    public void onAdClose() {
                        // 广告关闭时回调该方法
                        jumpToMain();
                    }
                })
                .build();
        ydSpread.requestSpread();
    }

/**
 * 先判断canJump的值，如果为false说明当前页面已不可见不进行跳转
 * 修改canJump值为true等再次返回此页面后再跳转
 */
private void jumpToMain() {
        if (canJump) {
            doJump();
        } else {
```

```
            canJump = true;
        }
}

/**
* 通过onPause()和onResume()方法和canJump变量，控制广告关闭后
* 以及从广告详情返回时页面的跳转
*/
@Override
protected void onPause() {
    super.onPause();
    canJump = false;
}

/**
* 通过onPause()和onResume()方法和canJump变量，控制广告关闭后
* 以及从广告详情返回时页面的跳转
*/
@Override
protected void onResume() {
    super.onResume();
    if (canJump) {
        doJump();
    }
    canJump = true;
}


@Override
protected void onDestroy() {
    super.onDestroy();
    if (ydSpread != null) {
        ydSpread.destroy();
    }
}

//禁止用户返回键退出APP，以免影响广告的曝光和计费
@Override
public boolean onKeyDown(int keyCode, KeyEvent event) {
    if (keyCode == KeyEvent.KEYCODE_BACK || keyCode == KeyEvent.KEYCODE_HOME) {
        return true;
    }
    return super.onKeyDown(keyCode, event);
}
```

备注：自定义跳过按钮样式时，不要将跳过按钮置于展示广告的容器内，否则会被移除。应该将跳过按钮与展示广告的容器平级设置，可参考Demo的布局方式

2. 插屏广告

插屏广告一般用于离开当前页时或者游戏角色死亡时弹出的广告，可自行传入宽高值，但最终插屏广告的宽高只能接近传入的数值，并不是1：1完全符合。请求插屏广告的时间相对较久（2s左右），为了更好的体验，所以请提前请求，展示的时候只需要show即可

```
YdInterstitial ydInterstitial;
   /**
* 请求插屏广告
```

```
* context 上下文
* setKey 运营分配的媒体位
* setWidth 请求广告的宽，单位dp，最终以实际宽度为准
* setHeight 请求广告的高，单位dp，最终以实际高度为准
* setIconListener 回调监听
*/
private void loadAd() {
        ydInterstitial = new YdInterstitial.Builder(this)
                .setKey(String)
                .setWidth(Integer)
                .setHeight(Integer)
                .setInterstitialListener(new AdViewInterstitialListener() {
                    @Override
                    public void onAdReady() {
                        //请求成功时回调。可以在此处调用展示
                         showAd();
                    }

                    @Override
                    public void onAdDisplay() {
                        //广告展示时回调
                    }

                    @Override
                    public void onAdClick() {
                        //广告被点击时回调
                    }

                    @Override
                    public void onAdClosed() {
                        //广告关闭时回调
                    }

                    @Override
                    public void onAdFailed(YdError error) {
                        //广告请求失败时回调
                    }

                })
                .build();
        ydInterstitial.requestInterstitial();
    }

// 展示广告。展示方法只在onAdReady回调后调用有效
private void showAd() {
    if (ydInterstitial != null && ydInterstitial.isReady()) {
         ydInterstitial.show();
    }
}

@Override
protected void onDestroy() {
    super.onDestroy();
    ydInterstitial.destroy();
}
```

3. Banner广告

Banner的宽度一般是屏幕的宽度，高度在100-150像素之间。Banner广告请求成功后，会有View返回，接入方将其展示在屏幕中即可

```
YdBanner ydBanner;
   /**
* 加载banner
* context 上下文
* setKey 运营分配的媒体位
* setBannerListener 回调监听
*/
private void loadBannerAd() {
        ydBanner = new YdBanner.Builder(Activity)
                .setKey(String)
                // 设置banner的宽度，单位dp，默认值为屏幕宽度，采用默认值时可不用设置
                // .setExpressWidth(int)
                // 设置banner的高度，单位dp，默认值为宽度1/6.4，采用默认值时可不用设置
                // .setExpressHeight(int)
                .setBannerListener(new AdViewBannerListener() {
                    @Override
                    public void onReceived(View view) {
                        // 请求广告成功，返回banner

                    }

                    @Override
                    public void onAdClick() {
                        // 广告被点击时回调该方法
                    }

                    @Override
                    public void onAdFailed(YdError error) {
                        // 广告加载出错时回调该方法
                    }
                })
                .build();
        ydBanner.requestBanner();
    }

@Override
protected void onDestroy() {
    super.onDestroy();
    ydBanner.destroy();
}
```

4. 激励视频

激励视频是一种收益较高的广告类型，广告播放过程中，不能中途关闭，只有广告播放完毕后才会出现关闭按钮，选择使用时需慎重

```
/**
* 请求激励视频
* @param String 运营分配的key
* @param AdViewVideoListener 回调
*/
```

```
private void loadRewardVideoAd() {
  ydVideo = new YdVideo.Builder(this)
    .setKey(String)
    .setUserId("唯一id")//不使用服务端校验无须设置，如使用服务端校验必须设置
    .setCustomData("自定义数据")//不使用服务端校验无须设置，如使用服务端校验选择设置
    .setAdViewVideoCheckListener(new AdViewVideoCheckListener() {//不使用服务端校验
无须设置
        @Override
        public void onVideoCheckReward(String transID) {
        //返回校验用订单id,如开启服务端校验，会在onVideoReward回调前给此回调，如未开启服务
端校验无需设   置此监听
        }
    })
    .setVideoListener(new AdViewVideoListener() {

      @Override
      public void onAdShow() {
            // 视频展示回调
      }

      @Override
      public void onAdClose() {
            // 视频关闭回调
      }

      @Override
      public void onVideoPrepared() {
            // 视频加载完成回调
            // 需要在此回调后调用视频播放方法
            showVideo();
      }

      @Override
      public void onVideoReward() {
            // 视频播放奖励回调
      }

      @Override
      public void onVideoCompleted() {
            // 视频播放完毕回调
      }

      @Override
      public void onAdClick() {
            // 视频点击回调
      }

      @Override
      public void onAdFailed(YdError error) {
            // 视频请求失败回调
      }
    })
    .build();
  ydVideo.requestRewardVideo();
}

/**
* 播放视频广告
```

```
*/

public void showVideo() {
  if (ydVideo != null && ydVideo.isReady()) {
    ydVideo.show();
  }
}

@Override
protected void onDestroy() {
    super.onDestroy();
    if (ydVideo != null) {
        ydVideo.destroy();
    }
}
```

5.全屏视频

全屏视频在广告形式上类似激励视频，区别在于广告播放开始或者几秒钟后可以选择关闭，全屏视频没有额外获取奖励的回调。

```
     private YdFullVideo ydFullVideo;

     ydFullVideo = new YdFullVideo.Builder(this)
                .setKey(String)//广告id
                .setVideoListener(new AdViewFullVideoListener() {

                    @Override
                    public void onAdFailed(YdError error) {
                       //请求失败时调用
                    }

                    @Override
                    public void onAdClicked() {
                       //广告点击时调用
                    }

                    @Override
                    public void onPageDismiss() {
                        //广告关闭时调用
                    }

                    @Override
                    public void onVideoPrepared() {
                        // 视频加载完成时调用

                        //可以在视频加载完成回调后择机展示
                        if (ydFullVideo != null && ydFullVideo.isReady()) {
                            ydFullVideo.show();//展示全屏视频
                        }
                    }

                    @Override
                    public void onVideoPlayStart() {
                         // 视频展示时调用
                    }
```

```
                })
                .build();
        ydFullVideo.requestFullVideo();//请求全屏视频广告
```

6.原生混合广告

原生混合广告是将**自渲染信息流广告**，**模版信息流广告**，**Draw信息流广告**封装到同一个API中的广告请求方式。开发者可以在同一个广告位中配置上述不同的广告样式混用。

1. API说明

广告请求

```
public class YdSDK {
    /**
     * 加载原生混合广告请求对象
     *
     * @param context  请使用Activity作为上下文
     * @param params   广告请求的参数配置
     * @param listener 广告加载回调监听
     */
    public static void loadMixNative(Context context, AdParams params,
NativeLoadListener listener)
}


/**
 * 原生混合广告请求的参数配置
 */
public class AdParams {

    /**
     * 原生混合广告配置构造器
     */
    public static class Builder {

        /**
         * 构造函数
         *
         * @param posId 广告位Id
         */
        public Builder(String posId)

        /**
         * 设置模板广告的期望宽度，优量汇、穿山甲、京准通、OPPO、Vivo等广告联盟需要。
         * 优量汇可配置为宽度适配父控件宽度，参见{@link #setExpressFullWidth}方法。
         *
         * @param width 模板广告的期望宽度，单位dp
         * @return {@link AdParams.Builder}
         */
        public Builder setExpressWidth(float width)

        /**
         * 设置模板广告的期望高度，优量汇、穿山甲、京准通、OPPO、Vivo等广告联盟需要。
         * 优量汇、穿山甲、Vivo支持高度自适应，参见{@link #setExpressAutoHeight}方法。
```

```
         *
         * @param height 模板广告的期望高度，单位dp
         * @return {@link AdParams.Builder}
         */
        public Builder setExpressHeight(float height)

        /**
         * 设置自渲染广告期望的图片素材宽度。穿山甲，京准通需要。
         *
         * @param width 渲染广告期望的图片素材宽度，单位px
         * @return {@link AdParams.Builder}
         */
        public Builder setImageAcceptedWidth(int width)

        /**
         * 设置自渲染广告期望的图片素材高度。穿山甲，京准通需要。
         *
         * @param height 渲染广告期望的图片素材高度，单位px
         * @return {@link AdParams.Builder}
         */
        public Builder setImageAcceptedHeight(int height)

        /**
         * 模板广告高度自适应，优量汇、穿山甲、Vivo支持。
         *
         * @return {@link AdParams.Builder}
         */
        public Builder setExpressAutoHeight()

        /**
         * 模板广告宽度最大程度的使用其父控件给予的宽度，优量汇支持。
         *
         * @return {@link AdParams.Builder}
         */
        public Builder setExpressFullWidth()

        /**
         * 生成{@link AdParams}对象。
         *
         * @return {@link AdParams}
         */
        public AdParams build()
    }
}


/**
 * 原生混合广告加载监听回调
 */
public interface NativeLoadListener {
    /**
     * 广告请求成功
     * @param nativeAd 请求成功的原生混合广告对象
     */
    void onNativeAdLoaded(NativeAd nativeAd);

    /**
     * 广告请求错误
```

```
     * @param error 错误原因
     */
    void onAdFailed(YdError error);
}
```

广告渲染

```
/**
 * 原生混合广告对象，广告请求成功返回此对象
 */
public interface NativeAd {
    /**
     * 是否为模板渲染类型的广告
     *
     * @return false：自渲染广告，ture：模板广告
     */
    boolean isNativeExpress();

    /**
     * 绑定展示广告的容器
     *
     * @param nativeAdView    展示广告的容器，广告必须在此容器中，必须为{@link
NativeAdView}
     * @param selfRenderView {@link #isNativeExpress()}返回false时（即自渲染广告）需
传入开发者自定义的广告view；
     *                        返回ture时（即模板广告）可传null。
     */
    void renderAdContainer(NativeAdView nativeAdView, View selfRenderView);

    /**
     * 渲染展示广告
     *
     * @param prepareInfo 自渲染广告需绑定广告组件配置，用于配置广告点击控件、关闭按钮等
等。
     *                    <p>
     *                    {@link #isNativeExpress()}返回ture时（即模板广告）可传
null。
     *                    对于模板广告，穿山甲广告需要处理关闭按钮必须设置Activity
     */
    void prepare(NativePrepareInfo prepareInfo);

    /**
     * 绑定视频控件的状态，在Activity的onResume时调用（主要针对部分广告平台的视频广告）
     */
    void onResume();

    /**
     * 绑定视频控件的状态，在Activity的onPause时调用（主要针对部分广告平台的视频广告）
     */
    void onPause();

    /**
     * 获取广告素材或广告view
```

```
     */
    NativeMaterial getAdMaterial();

    /**
     * 获取广告的ECPM
     *
     * @return ECPM值，单位：分。
     */
    int getECPM();

    /**
     * 设置广告事件监听
     * @param listener 广告事件监听器
     */
    void setNativeEventListener(NativeEventListener listener);

    /**
     * 销毁当前的广告对象（执行之后该广告无法再进行展示）
     */
    void destroy();
}


/**
 * 自渲染广告需绑定广告组件配置
 */
public class NativePrepareInfo {

    /**
     * 可为null，设置关闭按钮，仅穿山甲、京准通支持。设置后支持{@link
NativeEventListener}中的广告关闭回调。
     * 在支持的广告联盟渲染后调用方不可对其设置点击事件监听。
     *
     * @param closeView 关闭按钮
     */
    public void setCloseView(View closeView)

    /**
     * 设置创意点击区(Action)，点击事件将有SDK处理，渲染后调用方不可对其设置点击事件监听。
     *
     * @param ctaViews 创意点击区(Action)View
     */
    public void setCtaView(View... ctaViews)

    /**
     * 设置可点击View，点击事件将有SDK处理，渲染后调用方不可对其设置点击事件监听。
     *
     * @param clicks 可点击View
     */
    public void setClickView(View... clicks)

    /**
     * 设置图片展示控件
     *
     * @param views 图片展示控件
     */
    public void setImageView(View... views)
```

```
    /**
     * 设置展示广告时的Activity
     *
     * @param activity 广告时的Activity
     */
    public void setActivity(Activity activity)
}
```

广告事件监听

```
/**
 * 原生混合广告的事件监听回调
 */
public interface NativeEventListener {
    /**
     * 广告展示回调
     */
    void onAdImpressed(NativeAdView adView);

    /**
     * 广告点击回调
     */
    void onAdClicked(NativeAdView adView);

    /**
     * 广告视频播放开始
     */
    void onAdVideoStart(NativeAdView adView);

    /**
     * 用户点击关闭按钮，收到需调用方将广告View{@link NativeAdView}移除界面，部分广告联盟会自动移除广告View。
     * 模板广告均支持此回调，穿山甲广告渲染时需绑定Activity后支持；
     * 自渲染广告仅穿山甲、京准通绑定closeView后，支持此回调。
     */
    void onAdClose(NativeAdView adView);

    /**
     * 广告视频播放开始
     */
    void onAdVideoEnd(NativeAdView adView);

    /**
     * 广告视频播放进度，仅部分广告联盟支持
     */
    void onAdVideoProgress(NativeAdView adView, long progress);

    /**
     * 广告渲染异常或视频播放异常
     */
    void onAdFailed(NativeAdView adView, YdError error);
}
```

广告素材定义

```java
/**
 * 原生混合广告的素材对象
 */
public interface NativeMaterial {

    /**
     * 获取素材类型：
     * {@link NativeAdConst#AD_TYPE_UNKNOWN}：未知类型；
     * {@link NativeAdConst#AD_TYPE_SINGLE_IMG}：单图；
     * {@link NativeAdConst#AD_TYPE_GROUP_IMG}：组图；
     * {@link NativeAdConst#AD_TYPE_VIDEO}：视频。
     *
     * @return 素材类型
     */
    int getAdType();

    /**
     * 获取主图，可能为null，仅部分广告联盟有值。
     *
     * @return 图片的URL地址
     */
    @Nullable
    String getMainImageUrl();

    /**
     * 获取图片列表，部分广告联盟可能返回null或空列表。
     *
     * @return 图片的URL地址List
     */
    List<String> getImageUrlList();

    /**
     * 获取广告标题
     *
     * @return 广告标题
     */
    String getTitle();

    /**
     * 获取广告描述
     *
     * @return 广告描述
     */
    String getDescription();

    /**
     * 获取广告的创意文案，部分广告联盟可能返回null。
     *
     * @return 返回创意文案
     */
    String getCallToAction();

    /**
     * 获取广告的ICON图标URL地址，部分广告联盟可能返回null。
     *
```

```
     * @return 广告的ICON图标URL地址
     */
    String getIconUrl();

    /**
     * 获取广告的MediaView
     *
     * @return 自渲染广告返回视频的View，模板广告返回广告的View
     */
    View getAdMediaView();

    /**
     * 获取视频的URL地址，请优先使用{@link #getAdMediaView()}展示视频
     *
     * @return 视频的URL地址
     */
    String getVideoUrl();

    /**
     * 获取广告标示，可能为null，仅部分广告联盟有值
     *
     * @return 广告标示的Bitmap对象
     */
    Bitmap getAdLogo();

    /**
     * 获取广告标示，可能为null，仅部分广告联盟有值
     *
     * @return 广告标示的URL地址
     */
    String getAdLogoUrl();

    /**
     * 返回视频时长，不支持获取视频时长的广告联盟返回0。
     *
     * @return 视频时长
     */
    double getVideoDuration();
}
```

2. 接入代码示例

```
// 1. 构造请求参数
int width = DeviceUtil.px2dip(container.getWidth());
int imageW = DeviceUtil.dip2px(240);
int imageH = DeviceUtil.dip2px(130);

AdParams params = new AdParams.Builder(BuildConfig.MixNative)
                // 期望模板宽高，优量汇、穿山甲、京准通、OPPO、Vivo等广告联盟需要
                // 优量汇可配置为宽度适配父控件宽度，优量汇、穿山甲、Vivo支持高度自适应
                .setExpressHeight(100) // 期望模板高度
                .setExpressWidth(width) // 期望模板宽度
                .setExpressAutoHeight() // 模板高度自适应，优量汇、穿山甲、Vivo支持
                .setExpressFullWidth() // 模板宽度适配父控件宽度，优量汇支持
                // 自渲染广告期望的图片素材宽高，穿山甲，京准通需要
```

```
                .setImageAcceptedHeight(imageH)
                .setImageAcceptedWidth(imageW)
                .build();
// 2. 发起请求
YdSDK.loadMixNative(this, params, new NativeLoadListener() {
    @Override
    public void onNativeAdLoaded(NativeAd nativeAd) {
        // 3. 渲染请求成功广告
        inflateAd(nativeAd);
    }

    @Override
    public void onAdFailed(YdError error) {
        // 4. 请求失败处理
        String msg = String.format(Locale.getDefault(), "onAdFailed,
code:%d,msg:%s", error.getCode(), error.getMsg());
        Toast.makeText(MixNativeActivity.this, msg, Toast.LENGTH_LONG).show();
    }
});

// 渲染广告
private void inflateAd(NativeAd nativeAd) {
    setNativeEvent(nativeAd);

    FrameLayout container = findViewById(R.id.container_ad);
    container.removeAllViews();

    // 构造NativeAdViewd对象，广告View均需在此ViewGroup中
    NativeAdView nativeAdView = new NativeAdView(this);
    container.addView(nativeAdView, new FrameLayout.LayoutParams(WRAP_CONTENT,
WRAP_CONTENT));

    // 判断是否为模板广告
    if (nativeAd.isNativeExpress()) {
        // 对于模板广告只需获取MediaView填充到广告容器中即可
        View mediaView = nativeAd.getAdMaterial().getAdMediaView();
        nativeAdView.addView(mediaView, new
FrameLayout.LayoutParams(MATCH_PARENT, MATCH_PARENT));
        // 绑定模板广告容器，对于模板广告，第二参数selfRenderView可为null
        nativeAd.renderAdContainer(nativeAdView, null);
        NativePrepareInfo prepareInfo = new NativePrepareInfo();
        // 渲染模板需绑定Activity
        prepareInfo.setActivity(this);
        // 渲染模板广告
        nativeAd.prepare(prepareInfo);
        return;
    }

    // 以下为自渲染广告

    // 媒体自定义的广告View
    View adView = View.inflate(this, R.layout.ad, null);
    nativeAdView.addView(adView, new FrameLayout.LayoutParams(MATCH_PARENT,
MATCH_PARENT));

    // 绑定广告容器，第一个参数NativeAdView必须为广告的父容器；第二参数为自定义的广告view，
不可为空
    nativeAd.renderAdContainer(nativeAdView, adView);
```

```
    // 获取自定义广告的展示控件
    ImageView ivLogo = adView.findViewById(R.id.img_logo);
    TextView tvTitle = adView.findViewById(R.id.text_name);
    TextView tvDesc = adView.findViewById(R.id.text_desc);
    ImageView pic = adView.findViewById(R.id.img_poster);
    TextView btnOperate = adView.findViewById(R.id.btn_text);
    FrameLayout flMediaView = adView.findViewById(R.id.fl_mediaview);

    // 获取广告素材，并填充广告素材
    NativeMaterial nativeMaterial = nativeAd.getAdMaterial();
    tvTitle.setText(nativeMaterial.getTitle());
    tvDesc.setText(nativeMaterial.getDescription());
    if (!TextUtils.isEmpty(nativeMaterial.getIconUrl())) {

 Glide.with(getApplicationContext()).load(nativeMaterial.getIconUrl()).into(ivLo
go);
    }

    // 判断广告素材类型，对于视频广告需获取MediaView填充
    if (nativeMaterial.getAdType() == AD_TYPE_VIDEO &&
nativeMaterial.getAdMediaView() != null) {
        pic.setVisibility(View.GONE);
        flMediaView.addView(nativeMaterial.getAdMediaView(), new
FrameLayout.LayoutParams(MATCH_PARENT, MATCH_PARENT));
    } else {
        flMediaView.setVisibility(View.GONE);
        if (!TextUtils.isEmpty(nativeMaterial.getMainImageUrl())) {

 Glide.with(getApplicationContext()).load(nativeMaterial.getMainImageUrl()).into
(pic);
        } else {
            String imgUrl =
Check.findFirstNonNull(nativeMaterial.getImageUrlList());
            if (!TextUtils.isEmpty(imgUrl)) {
                Glide.with(getApplicationContext()).load(imgUrl).into(pic);
            }
        }
    }

    if (nativeMaterial.getAdLogo() != null) {
        adLogo.setImageBitmap(nativeMaterial.getAdLogo());
    } else if (!TextUtils.isEmpty(nativeMaterial.getAdLogoUrl())) {

 Glide.with(getApplicationContext()).load(nativeMaterial.getAdLogoUrl()).into(ad
Logo);
    }

    // 准备渲染广告时需要绑定的控件
    NativePrepareInfo prepareInfo = new NativePrepareInfo();
    prepareInfo.setActivity(this);
    prepareInfo.setCtaView(btnOperate);
    prepareInfo.setClickView(adView);
    prepareInfo.setImageView(pic);

    // 渲染广告
    nativeAd.prepare(prepareInfo);
}
```

```java
private void setNativeEvent(NativeAd nativeAd) {
    // 设置广告事件监听
    nativeAd.setNativeEventListener(new NativeEventListener() {
        @Override
        public void onAdImpressed(NativeAdView adView) {
            // 广告展示成功
        }

        @Override
        public void onAdClicked(NativeAdView adView) {
            // 广告被点击
        }

        /**
         * 用户点击关闭按钮，收到需调用方将广告View{@link NativeAdView}移除界面，部分广告联盟会自动移除广告View。
         * 模板广告均支持此回调，穿山甲广告渲染时需绑定Activity后支持；
         * 自渲染广告仅穿山甲、京准通绑定closeView后，支持此回调。
         */
        @Override
        public void onAdClose(NativeAdView adView) {
            // 广告被关闭，
        }

        @Override
        public void onAdVideoStart(NativeAdView adView) {
            // 视频播放开始
        }

        @Override
        public void onAdVideoEnd(NativeAdView adView) {
            // 视频播放结束
        }

        @Override
        public void onAdVideoProgress(NativeAdView adView, long progress) {
            // 视频播放进度，仅部分广告联盟支持
        }

        @Override
        public void onAdFailed(NativeAdView adView, YdError error) {
            // 广告渲染异常
        }
    });
}
```

7. 自定义广告源广告

详细实现自定义Adapter的示例可查看Demo

1. 针对实现各个广告形式需要继承的类说明

开发者需要根据自身需求继承相应广告形式的类（参照下表）并重写所有抽象方法，在相应的方法中调用广告平台的API，并通过类成员变量mLoadListener回调加载的结果，通过类成员变量mImpressionEventListener回调广告展示、点击、关闭等事件结果

广告形式	类名
激励视频	com.yd.saas.base.custom.rewardvideo.CustomRewardVideoAdapter
插屏广告	com.yd.saas.base.custom.Interstitial.CustomInterstitialAdapter
全屏视频	com.yd.saas.base.custom.fullvideo.CustomFullVideoAdapter
横幅广告	com.yd.saas.base.custom.banner.CustomBannerAdapter
原生模板	com.yd.saas.base.custom.template.CustomTemplateAdapter
原生自渲染	com.yd.saas.base.custom.nativead.CustomNativeAdapter
开屏广告	com.yd.saas.base.custom.spread.CustomSpreadAdapter

2. Adapter的使用以及注意事项

2.1 使用流程

- 开发者只需要继承相关类并重写相关方法，**自定义Adapter中的所有方法都不需要开发者进行调用**，当AD后台的广告位下配置好自定义广告平台的广告源后，**SDK内部会在合适的时机调用自定义Adapter中的相关方法。**
- 开发者调用AD SDK的load API加载广告，AD SDK会根据后台配置的类名通过反射创建出自定义Adapter的类，并调用**loadCustomNetworkAd()**进行广告的请求，自定义广告平台的广告加载结果需要通过调用CustomLoadListener的方法通知AD SDK，最终AD SDK会通过AD的回调方法通知开发者
- 自定义广告平台加载广告时所需要的参数（AppID、AppKey、PlacementID等）需以Json字符串的形式配置在AD后台的该广告平台的广告源中，例如：

  ```
  {
      "appId":"xxxxx",
      "placeId":"xxxxxxxxx"
  }
  ```

  AD SDK拉取到后台策略时，会将此Json字符串转化为Map，传给**loadCustomNetworkAd(Context context, Map<String, Object> serverExtra, Map<String, Object> localExtra)**方法，开发者可通过serverExtra参数获取到相关参数（AppID、AppKey、PlacementID等）
- 开发者调用AD SDK的isAdReady API时，会调用到自定义Adapter的isAdReady()方法
- 开发者调用AD SDK的show API时，会调用到自定义Adapter的show()方法

2.2 注意事项

- AD后台添加自定义广告平台时，**Adapter类名需配置全路径类名（包名+类名）**，否则加载广告时Adapter类将不能正常被创建
- 因为SDK内部是通过反射new出Adapter的类，所以在构建release包时，**必须确保Adapter类不能被混淆。**混淆规则示例如下：

```
# 将 <Adapter类名> 改为开发者自己的Adapter类名
-keep class <Adapter类名> { *;}
-keepclassmembers public class <Adapter类名> {
   public *;
}
```

8. 预加载广告

2.6.4以上版本支持广告预加载功能。提升广告响应速度。

使用方法：需要在主activity页面，调用相应方法，对广告进行预加载。

接入代码示例

```
YdConfig.getInstance().setActivityAndPreloadAd(this);
```

三、混淆

以下是所有的混淆规则，但需要根据实际所依赖的aar进行选择配置：

```
-keep class com.yd.** {*;}
```

四、国际版集成说明

1. 初始化

在调用SDK初始化方法 YdConfig.getInstance().init()前务必先调用YdSDK.setInternationalLocale()方法设置运行环境。

```
// 配置国际化运行环境，务必最先调用
YdSDK.setInternationalLocale();
// SDK 初始化在后
YdConfig.getInstance().init()
```

2. 配置GDPR

接入方需自行判断用户是否在欧盟地区或是否需要设置GDPR。SDK提供两种方式配置GDPR：

1. 不使用SDK提供的GDPR授权页，直接设置GDPR等级，接入方需自行处理授权页及GDPR状态；
2. 使用SDK提供的GDPR授权页。

```
// GDPR等级
YdGDPRAuth.GDPR_UNKNOWN; // 未知等级
YdGDPRAuth.GDPR_AGREE; // 已授权上报设备信息
YdGDPRAuth.GDPR_REFUSE; // 拒绝上报设备信息

// 方式1，在需要设置GDPR等级时，通过YdSDK.getGDPRLevel()方法获取SDK记录的等级，
// 当为GDPR_UNKNOWN等级时，应展示授权页设置页，并通过YdSDK.setGDPRLevel()方法设置授权结果
```

```
// 获取SDK记录的GDPR等级
YdSDK.getGDPRLevel();
// 直接设置GDPR等级
YdSDK.setGDPRLevel();


// 方式2，是否需要展示授权页同方式1，当需要展示授权页时通过调用YdSDK.showGdprAuth()方法打开授权页，
// 结果通过YdGDPRAuth.AuthReslut回调给接入方。
YdSDK.showGdprAuth(Activity, YdGDPRAuth.AuthReslut{
    // 用户授权结果
    void onAuthFinish(int reslut);

    // 授权页加载失败
    void onPageLoadFail();
});
```

五、竞价结果回传

YdSpread,YdInterstitial,YdBanner,YdVideo,YdFullVideo,NativeAd
调用biddingResultUpload(boolean isWinner, int price, int secondPrice)方法回传竞价结果

六、获取ecpm

YdSpread,YdInterstitial,YdBanner,YdVideo,YdFullVideo,NativeAd通过getECPM方法可以获取，单位为分。

七、隐私信息控制设置

```
YdConfig.getInstance().setPersonalizedState(false);//是否支持个性化广告
YdConfig.getInstance().setEnableCollectAppInstallStatus(false);//是否允许获取应用安装列表
YdConfig.getInstance().init(this, BuildConfig.APIKey, "",
        new YdParamConfig.Builder()
                .setCanUseAndroid(false)//是否允许获取AndroidId
                .setCustomAndroidId("测试自定义androidID")
                .setCanUseMac(false)//是否允许获取Mac地址
                .setCustomMac("测试自定义mac")
                .setCanUseIMEI(false)//是否允许获取IMEI
                .setCustomIMEI("测试自定义imei")
                .setCanUseIMSI(false)//是否允许获取IMSI
                .setCanUseBootID(false)//是否允许获取BootID
                .setCanUseLocation(false)//是否允许获取Location
                .build(), true);//YdParamConfig中各项设置参数true为允许获取，false为不获取
```

八、备注

demo仅供参考，具体以文档为准。如有疑问，请联系Ad³平台开发人员，谢谢~